

FlowEvo: Self-Evolving Agents through the Co-Evolution of Workflows and Executable Skills

Zeyu Ren^{1*} Ling Yue^{2*} Ran Li³ Yishu Wang¹ Shengxiang Xu¹
Hanmo Liu³ Shaowu Pan^{2†} Shimin Di^{1†}

¹Southeast University

²Rensselaer Polytechnic Institute

³The Hong Kong University of Science and Technology

Abstract

Large language model agents increasingly solve complex tasks by constructing inference-time workflows that combine reasoning, tool use, and code execution. While such workflows enable flexible problem solving, the useful procedures discovered during execution are often transient: they help solve the current task but are not retained in a form that can systematically benefit future tasks. We present FlowEvo, a training-free framework that compiles successful traces into reusable skill records. Each record pairs a callable artifact with auxiliary structured guidance, and admission applies interface, replay, and safety checks where feasible. These skill records persist in a skill bank at inference time. FlowEvo is organized around three coupled mechanisms: (1) workflow-to-skill compilation, which extracts reusable executable artifacts from successful traces; (2) skill-to-workflow feedback, which retrieves accumulated skills to support future problem solving through either direct execution or structured context injection; and (3) skill curation, which monitors downstream utility and suppresses skills that cause negative transfer. Through this workflow-skill-workflow feedback loop, FlowEvo enables agents to accumulate and refine task-solving capability over time without updating model parameters. Experiments on benchmarks spanning interactive environments (ALFWorld) and code/math generation (HumanEval, GSM8K) show that FlowEvo achieves the best accuracy-cost tradeoff among the evaluated baselines under our implementation settings. On ALFWorld, FlowEvo achieves an 82.8% success rate, 23.6 percentage points above the strongest baseline, while its average token usage per episode is less than half that of the most efficient baseline. Controlled ablations confirm that each mechanism contributes to the overall result. The code is public at <https://github.com/DEFENSE-SEU/FlowEvo>.

1 Introduction

Large language model (LLM) agents increasingly solve tasks by constructing workflows at inference time rather than following a fixed prompt or a fixed sequence of tool calls (Hu et al., 2024; Zhang et al., 2024; Wang et al., 2026a; Gao et al., 2025; Wang et al., 2025c). In these systems, solving a task may involve decomposition, tool use, code generation, intermediate checking, and repair (Wang et al., 2026b; Xu et al., 2025a; Zheng et al., 2025b). Such workflow-based agents are being deployed in diverse scientific and engineering domains, including computational fluid dynamics simulation (Pan et al., 2026), Earth observation (Yao et al., 2026), and even automated scientific peer review (Wang et al., 2025a). This flexibility has led to clear gains across coding and reasoning tasks. At the same time, this paradigm leaves an obvious inefficiency: useful procedures discovered during one episode are often not retained in a form that can be reliably reused later (Zhao et al., 2024).

*Equal contribution.

†Corresponding authors: pans2@rpi.edu, shimin.di@seu.edu.cn.

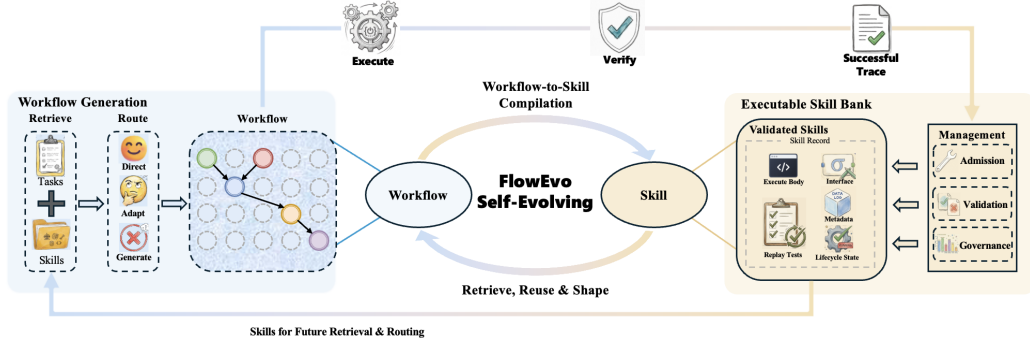


Figure 1: Overview of FlowEvo. Successful workflows are compiled into executable skills and stored in a persistent bank. Retrieved skills can either be executed directly or supplied as structured context for subsequent workflow generation.

This matters for two reasons. First, agents may repeatedly rediscover similar routines across related tasks, which increases token cost and introduces avoidable variance (Sumers et al., 2023). Second, a successful workflow often contains reusable operational structure, such as a useful decomposition pattern, a stable code scaffold, a verification step, or a repair strategy. If this structure is kept only as a transient trajectory, then later tasks still have to be solved largely from scratch (Zhu et al., 2023).

Existing work addresses parts of this problem, but the overall picture remains incomplete and still somewhat fragmented. One line of work stores prior plans, traces, or workflows as text and uses them to guide later generation (Wang et al., 2024b; Zhang et al., 2026b; 2025b; Park et al., 2023). Another line studies reusable tools, procedures, or skills that can be executed later (Zheng et al., 2025a; Chen et al., 2026; Qian et al., 2023; Wang et al., 2023). In practice, successful workflows lie between these two views. They contain more executable structure than text-based memory, but they are also less fixed than hand-designed tools. This raises a basic question: when an agent solves tasks through workflows, how should successful workflows be turned into reusable skills, and how should those skills affect later workflow construction when model parameters remain fixed?

We study this question through **FlowEvo**, a training-free framework for agent self-evolution through inference-time skill accumulation. When the agent solves a task successfully, the useful part of the workflow should not be kept only as a textual record. Instead, it should be compiled into a reusable skill record that pairs a callable artifact with auxiliary structured guidance, subject to interface, replay, and safety checks at admission where feasible, and stored in a persistent skill bank. Later, retrieved skills can help in two different ways. If a new task is sufficiently compatible with a stored skill, the skill can be executed directly. If direct execution is not appropriate, the retrieved skill can still be provided as structured context to support the construction of a new workflow. In this sense, FlowEvo treats self-evolution as a change in the agent’s inference-time capability layer.

This design creates a workflow–skill–workflow loop. Workflow execution produces successful traces. Successful traces are compiled into executable skills. The accumulated skill bank then changes how later tasks are solved. The first effect is *direct skill reuse*, where a validated callable artifact is executed on a new task. The second effect is *skill-conditioned workflow generation*, where retrieved skills help shape planning and generation even when they are not replayed directly. Distinguishing these two effects is important, because an improvement from skill accumulation should not be reduced to exact replay alone.

We evaluate FlowEvo on benchmarks spanning two task families: an interactive environment (ALFWorld; Shridhar et al., 2020) and code/math generation (HumanEval, Chen et al., 2021; GSM8K, Cobbe et al., 2021). We compare against both interactive-environment baselines (Reflexion, ExpEL) and automated workflow-optimization methods (AFLOW, ADAS),

and use controlled ablations on ALFWorld to isolate the contribution of each mechanism: workflow-to-skill compilation, skill-to-workflow feedback, and skill curation.

Our contributions are as follows:

- **Workflow-to-skill compilation.** We formulate skill acquisition as an online compilation step that turns successful workflows into reusable skill records, each combining a callable artifact with auxiliary structured guidance.
- **Tiered skill-to-workflow reuse.** We introduce a reuse mechanism that routes each task to one of three modes: dynamic generation, direct skill execution, or skill-conditioned workflow generation, with direct execution falling back to skill-conditioned generation when preconditions or validation fail.
- **Contrastive-utility lifecycle.** We develop a lifecycle that tracks each skill’s observational utility and automatically suppresses skills exhibiting persistently negative transfer.

2 Related Work

Workflow generation and optimization. A growing line of work studies agents that construct workflows dynamically at inference time rather than following a fixed sequence of prompts or tool calls (Yue et al., 2026a; Yuksekgonul et al., 2024; Khattab et al., 2023; Zhu et al., 2025). Within this setting, some methods search directly over workflow structure or optimize workflow graphs offline (Zhang et al., 2024; Hu et al., 2024; Zhang et al., 2025a). These approaches establish the importance of workflow structure for downstream performance, but they usually focus on finding better workflows for the current task or benchmark. Related efforts have distilled agent experience into reusable skills (Wang et al., 2023), insights (Zhao et al., 2024), or subroutines (Wang et al., 2024b), but remain tied to specific task domains. Our focus is different: we ask how successful workflows can become reusable artifacts that remain available after the original task has finished.

Memory for workflow agents. Another line of work augments agents with memories of prior plans, traces, or routines (Zhang et al., 2025c; Xu et al., 2025b). Agent Workflow Memory (Wang et al., 2024b) is the closest example in spirit, as it stores workflow-level experience to guide later generation. Text-based memories can improve planning and reduce repeated search, but they only describe prior behavior rather than provide directly executable procedures. This limitation motivates our decision to convert successful workflows into reusable skill records that combine callable artifacts with auxiliary structured guidance (Wang et al., 2025b; Xia et al., 2026b).

Skills, tools, and executable reuse. A separate literature studies reusable procedures, tools, APIs, and skills as first-class objects for agent systems, showing that executable artifacts can be more verifiable and reusable than text-only memories (Cai et al., 2023; Qian et al., 2023; Yuan et al., 2023; Wang et al., 2024a; Zheng et al., 2025a; Ouyang et al., 2025; Di et al., 2026). However, these methods typically construct tools for a fixed task distribution (Yuan et al., 2023; Wang et al., 2024a) or treat tool creation as a standalone step decoupled from the agent’s own problem-solving trajectory (Qian et al., 2023; Cai et al., 2023; Xia et al., 2026a). FlowEvo instead treats skill acquisition as a compilation step applied to the agent’s own successful workflows, closing the loop between dynamic workflow generation and persistent executable reuse.

Training-free adaptation and self-evolution. Recent work has also explored how agents improve over time without updating model weights, using search, self-generated feedback, memory revision, or evolving control state (Madaan et al., 2023; Putta et al., 2024; Yuan et al., 2024; Zhang et al., 2026a;b; Tian et al., 2024; Yue et al., 2025). In interactive environments, CLIN (Prasad Majumder et al., 2023) maintains a persistent memory of causal abstractions that is updated after each trial, enabling continual cross-task improvement in the same regime. This perspective is close to ours in spirit, but the evolving object in FlowEvo is more specific and operationally grounded: an executable skill bank distilled from successful

workflows. Our emphasis is therefore not on general online adaptation, but rather on a concrete mechanism for capability accumulation at inference time.

Position of this work. Our work targets a central unresolved problem across these lines of research: how a workflow-based agent can persistently accumulate reusable capability from its own successful executions, rather than repeatedly rediscovering similar procedures from scratch. Existing workflow-optimization methods improve behavior on the current task, but usually do not preserve what has been learned in an executable form. Memory-based methods retain prior experience, but primarily as text, which can guide planning yet cannot be directly verified or invoked. Skill- and tool-centric methods enable executable reuse, but are often designed for fixed domains or constructed outside the agent’s own online problem-solving loop. Training-free self-evolution methods, meanwhile, study adaptation at a broader level without providing a concrete mechanism for turning successful experience into persistent operational capability. FlowEvo addresses this gap by compiling successful workflows into reusable skill records that combine callable artifacts with auxiliary structured guidance, and by feeding them back into future problem solving through both direct reuse and skill-conditioned workflow generation, enabling agents to accumulate task-solving capability at inference time.

3 Method

3.1 Problem setup and the workflow-to-skill-to-workflow loop

Consider a stream of tasks $\{x_t\}_{t=1}^T$. At episode t , the agent operates with an inference-time context

$$C_t = (B_t, M_t, R_t), \quad (1)$$

where B_t is the executable skill bank, M_t denotes auxiliary memory layers, and R_t collects retrieval, routing, and failure-aware state accumulated from prior use.

Given task x_t , FlowEvo retrieves candidate artifacts, selects a routing mode, constructs a workflow plan when needed,

$$W_t = G(x_t; C_t), \quad (2)$$

executes and verifies the resulting program,

$$(\tau_t, y_t) = E(x_t, W_t), \quad (3)$$

and, when a successful episode yields reusable structure, compiles a candidate executable artifact and updates the memory state,

$$(B_{t+1}, M_{t+1}, R_{t+1}) = U(B_t, M_t, R_t, \tau_t, y_t). \quad (4)$$

Self-evolution therefore occurs through updates to executable and non-executable memory, with the base model held fixed.

The central design choice in FlowEvo is that successful workflows are not retained only as text. Instead, they are compiled into executable skills that can later affect new tasks in two distinct, practically important, and complementary ways. First, a retrieved skill may be executed directly when compatibility and validation make direct reuse reliable enough. Second, the same stored artifact may be exposed only as structured context that guides the generation of a new workflow. FlowEvo therefore treats the skill bank as a persistent inference-time capability layer rather than as prompt text alone.

Algorithm 1 gives the per-episode procedure, showing how retrieval, routing, compilation, admission, and skill curation are ordered within one inference-time loop.

System decomposition. Operationally, FlowEvo consists of four tightly coupled components: a workflow planner, an executor with verification, a trace-to-skill compiler, and a skill registry with curation. The planner proposes a route and constructs a workflow conditioned on the current task and retrieved artifacts. The executor runs the resulting program and returns a structured trace together with corresponding verification outcomes. The compiler

Algorithm 1 FlowEvo workflow–skill–workflow loop at inference time

Require: Task stream $\{x_t\}_{t=1}^T$; fixed base model θ

- 1: Initialize skill bank $B_0 \leftarrow \emptyset$; auxiliary memory $M_0 \leftarrow \emptyset$; routing state $R_0 \leftarrow \emptyset$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: $\mathcal{S}_t \leftarrow \text{RETRIEVE}(x_t, B_t, R_t)$ $\triangleright$ score candidates by pattern, interface, and utility
- 4: $r_t \leftarrow \text{ROUTESELECT}(x_t, \mathcal{S}_t)$ $\triangleright$ dynamic / direct-execute / skill-conditioned
- 5: **if** $r_t = \text{direct-execute}$ **and** $\text{COMPATIBLE}(x_t, \mathcal{S}_t)$ **then**
- 6: $W_t \leftarrow$ call retrieved skill $s^* \in \mathcal{S}_t$ as an executable subroutine
- 7: **else if** $r_t = \text{skill-conditioned}$ **then**
- 8: $W_t \leftarrow G(x_t; C_t = (B_t, M_t, R_t))$ $\triangleright$ generate with structured skill context
- 9: **else**
- 10: $W_t \leftarrow G(x_t; \emptyset)$ $\triangleright$ dynamic generation without skill context
- 11: **end if**
- 12: $(\tau_t, y_t) \leftarrow E(x_t, W_t)$ $\triangleright$ execute and verify; y_t indicates success
- 13: **if** $y_t = \text{success}$ **then**
- 14: $\hat{s}_t \leftarrow \text{COMPILE}(\tau_t)$ $\triangleright$ extract entry point, interface, replay tests, metadata
- 15: $B_{t+1} \leftarrow \text{ADMIT}(B_t, \hat{s}_t)$ $\triangleright$ duplicate check; interface, replay, safety validation
- 16: **else**
- 17: $B_{t+1} \leftarrow B_t$
- 18: **end if**
- 19: $M_{t+1}, R_{t+1} \leftarrow \text{UPDATE}(M_t, R_t, \tau_t, y_t, \mathcal{S}_t)$ $\triangleright$ update usage stats, negative evidence
- 20: $B_{t+1} \leftarrow \text{CURATE}(B_{t+1}, R_{t+1})$ $\triangleright$ suppress skills with persistently negative contrastive delta
- 21: **end for**
- 22: **return** final skill bank B_{T+1}

extracts reusable executable artifacts only from verifier-passed successful episodes. The registry maintains retrieval indices, admission state, and lifecycle updates for stored skills. This decomposition makes the workflow-to-skill-to-workflow loop directly observable at the trace level and helps attribute downstream gains to specific stored skills rather than to untracked prompt history alone.

3.2 Skill representation

A skill record in FlowEvo is a structured artifact that pairs an executable component with auxiliary textual guidance, rather than a free-form text note. The executable component is subject to interface, replay, and safety checks at admission where feasible, while the textual guidance supports skill-conditioned generation when direct execution is not selected. At a high level, we represent each skill abstractly as

$$s = (f, \sigma, \mathcal{T}, m, \ell), \quad (5)$$

where f is the executable body together with an identified entry point, σ is an explicit callable interface, $\mathcal{T}$ is a set of replay or verifier-style tests, m stores associated metadata, and ℓ denotes the current lifecycle state.

The metadata makes skill reuse explicit and auditable in practice. In the main system, it includes source information, task-pattern information, reuse constraints, fallback rules, and lightweight routing statistics. This representation allows a retrieved skill to be called through a known interface, checked against stored validation data, and systematically tracked over time as its downstream behavior changes.

The same stored artifact supports two reuse modes. For direct execution, the system relies on the interface, preconditions, direct-eligibility status, and replay evidence associated with the skill. For skill-conditioned workflow generation, the system exposes a compact structured view of the same artifact, such as its signature, task pattern, scaffold hint, or a short code excerpt. The same persistent object therefore supports both execution and planning, but the two uses have different reliability requirements and are evaluated separately.

Concretely, the metadata includes provenance (source task and fold), task-pattern tags, direct-reuse eligibility, preferred usage mode, fallback constraints, and lightweight negative

Table 1: Routing modes in FlowEvo. The distinction between direct execution and skill-conditioned generation is central to the mechanism analysis.

Route	Role of retrieved skill	Executes code	Indirect effect
Dynamic generation	No retrieved skill is used.	✗	✗
Direct skill execution	A retrieved skill is called as an executable subroutine before verification.	✓	✗
Skill-conditioned generation	Retrieved skills are supplied only as structured context for planning and generation.	✗	✓

evidence used to down-weight previously harmful skills. These fields are not all exposed in every route, but they make reuse decisions auditable and allow the same stored artifact to support both direct execution and workflow conditioning.

3.3 Workflow generation with retrieved skill context

For each task, a retriever scores candidate skills using lexical overlap, task-pattern compatibility, callable-interface compatibility, and historical utility signals derived from prior usage. Retrieval is also explicitly aware of negative evidence so that previously harmful or mismatched skills are penalized rather than treated as fresh candidates.

For clarity, we group the planner’s behavior into three main routes: (i) dynamic generation with no retrieved skill, (ii) direct execution of a retrieved skill, and (iii) skill-conditioned generation in which retrieved skills are supplied only as structured context. The implementation contains finer-grained seeded modes inside the third branch, but Table 1 captures the main distinction used in the experiments.

Direct execution is permitted only for skills that remain compatible after filtering and validation. Skill-conditioned generation is used when retrieval is informative enough to guide planning but not reliable enough for direct execution, or when direct execution is intentionally disabled for analysis purposes. In this route, the planner typically injects a compact structured view of the skill rather than replaying the full artifact, which limits prompt overhead and makes it easier to more clearly attribute gains to structured reuse rather than unrestricted replay of prior artifacts.

3.4 Compiling successful workflows into executable skills

When a task succeeds, FlowEvo compiles the verified trace into a reusable executable artifact. Rather than merely storing a textual summary, the compiler packages the successful candidate code into a reusable object: it identifies an entry point, infers a callable signature, extracts replay-style tests when possible, records source and task-pattern metadata, and stores routing-oriented metadata such as reuse constraints and preferred usage mode. The compiler thus converts episode-level success into a persistent executable capability together with a reusable contract for later routing.

In the implementation, compilation is attempted only from verifier-passed successful episodes. The compiler extracts a callable entry point, infers a lightweight interface, packages replay-style tests when available, and emits a candidate skill with an initial usage subtype. This candidate is then passed to the admission layer, which may retain it for direct execution, retain it only for context exposure, or leave it in a shadow state for diagnostics.

This compilation step is the link between transient workflow execution and persistent skill accumulation. The result is not only a summary of what happened during solving; it is a callable artifact that can be replayed, retrieved, and validated on later tasks.

3.5 Admission and skill curation

Not every successful trace should enter the active bank directly. Each candidate skill is first checked for duplicates and then passed through an admission layer before it becomes eligible for retrieval. Admission is governed by three diagnostic dimensions. The first is *interface compliance*: whether the extracted skill can be called under the intended signature and preconditions. The second is *functional correctness*: whether replay tests and held-out checks support the behavior claimed by the skill record. The third is *safety compliance*: whether the skill respects basic deployment constraints such as banned imports and banned calls (Appendix B). Depending on the outcome, a skill may be admitted for direct use, retained only for skill-conditioned use, or kept in a shadow state for subsequent diagnostics. Skills that fail interface, replay, or safety checks are not exposed for direct execution, and borderline cases may be retained only for skill-conditioned use or shadowed for diagnosis.

Once admitted, skills remain subject to lifecycle control based on their *downstream utility*—whether the skill improves later solving rather than merely existing in the bank. FlowEvo tracks downstream utility, failure patterns, and audit outcomes, and can disable, restrict, repair, or prune a skill when repeated negative transfer accumulates. In the current implementation, these policies are conservative: targeted audit episodes are used to re-check suspicious skills, and direct-use privileges can be downgraded to skill-conditioned-only use before removal. Unlike the three admission dimensions above, downstream utility cannot be evaluated at admission time and is instead assessed continuously after the skill enters the bank.

A key mechanism for detecting negative transfer is *contrastive evaluation*. For each skill that is actively used, FlowEvo periodically compares the success rate of episodes where the skill was applied against matched episodes where it was withheld, yielding a contrastive delta. When this delta is persistently negative, the skill is flagged for suppression. This provides a lightweight causal signal for identifying harmful skills without requiring explicit holdout experiments or ground-truth labels for individual skill contributions.

4 Experiments

We evaluate FlowEvo along two axes. First, we compare against interactive-environment baselines and automated workflow-optimization methods to establish cross-domain effectiveness in task accuracy and inference efficiency (Section 4.2). Second, we use controlled ablations on ALFWorld to isolate the contribution of each framework mechanism: workflow-to-skill compilation, skill-to-workflow feedback, and skill curation (Section 4.3).

4.1 Experimental setup

Model and inference. All experiments use GPT-4o-mini as the fixed base model accessed via API. FlowEvo evolves only through its inference-time capability layer (the executable skill bank, auxiliary memories, and routing statistics) without any parameter updates.

Benchmarks. We evaluate on benchmarks spanning two task families. (1) *Interactive environment*: ALFWorld (Shridhar et al., 2020), a text-based household task environment with 134 evaluation tasks from the eval_out_of_distribution split covering six task types. The agent interacts with the environment step by step (up to 50 steps per episode) and receives binary success feedback. (2) *Code and math generation*: HumanEval (Chen et al., 2021) (164 problems) and GSM8K (Cobbe et al., 2021) (1,319 test problems). For ALFWorld we report success rate; for HumanEval we report pass@1; for GSM8K we report solve rate. We also report average token consumption per task to assess inference efficiency.

Baselines. We compare against four representative methods spanning two paradigms. Self-refinement methods: Reflexion (Shinn et al., 2023) (self-reflective retry with up to 3 trials) and ExpeL (Zhao et al., 2024) (experience learning from prior trajectories). Workflow-optimization methods: AFLOW (Zhang et al., 2024) (MCTS-based workflow optimization) and ADAS (Hu et al., 2024) (automated workflow search). All baselines are implemented

Table 2: Main results across three benchmarks (GPT-4o-mini). Bold = best accuracy or lowest tokens; underline = second best.

Method	ALFWorld		HumanEval		GSM8K	
	SR (%)	Tokens	pass@1 (%)	Tokens	Solve (%)	Tokens
Reflexion	52.2	31,900	92.7	890	<u>96.8</u>	555
ExpeL	46.3	32,958	89.0	<u>883</u>	92.1	<u>552</u>
ADAS	53.0	29,671	82.4	3,776	90.8	2,820
AFLOW	<u>59.2</u>	<u>30,137</u>	<u>94.7</u>	3,826	93.5	2,494
FlowEvo	82.8	12,267	95.1	880	97.1	541

and evaluated under matched conditions using the same GPT-4o-mini API configuration. For AFLOW and ADAS, which were originally evaluated on different benchmark suites, we implement their workflow architectures following their published descriptions and open-source code, adapting them to each benchmark as needed.

FlowEvo configuration. Each benchmark is run as an independent experiment from an empty initial skill bank; no skills or memories are transferred across benchmarks. For ALFWorld, the agent accumulates skills online as it processes the 134 evaluation tasks in sequence. For code and math benchmarks, FlowEvo combines skill compilation with adaptive escalation: each task is first attempted with a single deterministic generation, and more expensive strategies (alternative sampling, skill-conditioned generation, reflective retry) are engaged only when earlier stages fail verification.

4.2 Main results

Table 2 presents the main comparison across three benchmarks, reporting both task accuracy and average token consumption per task. FlowEvo achieves the highest accuracy and the lowest inference cost on all three benchmarks simultaneously.

Interactive environments. On ALFWorld, FlowEvo attains 82.8%, outperforming AFLOW by 23.6 percentage points and Reflexion by 30.6 points. This large margin arises because FlowEvo compiles successful action sequences into parameterized templates that can be directly replayed on structurally similar tasks, whereas other methods must often re-derive solutions largely from scratch. FlowEvo simultaneously reduces average token usage to 12,267 per episode, less than half the cost of any baseline, since direct skill reuse bypasses much of the exploratory interaction process.

Code generation. On HumanEval, FlowEvo reaches 95.1%, surpassing AFLOW (94.7%) and Reflexion (92.7%), while consuming only 880 tokens per task. By contrast, AFLOW and ADAS rely on fixed multi-stage pipelines. AFLOW typically converges to an MCTS-optimized graph comprising five stages: generation, testing, ensembling, review, and revision. ADAS employs a meta-agent-searched workflow of comparable depth. Both pipelines incur substantial cost, 3,826 and 3,776 tokens per task respectively, yet achieve lower accuracy than FlowEvo. FlowEvo avoids this overhead through adaptive escalation: most tasks are solved in a single generation pass, with more expensive strategies engaged only on verification failure.

Mathematical reasoning. On GSM8K, FlowEvo reaches 97.1% while consuming 541 tokens per problem. Among the baselines, Reflexion is closest in both accuracy (96.8%) and token cost (555); AFLOW and ADAS reach lower accuracy (93.5%, 90.8%) at 2,494 and 2,820 tokens per problem, because they apply a fixed multi-stage pipeline to every problem regardless of difficulty. The mechanism question on GSM8K is distinct from ALFWorld: each problem is effectively unique, so the direct-reuse route contributes little and the gain cannot come from the same source as in the interactive setting. Two mechanisms account for the result. First, adaptive escalation solves most problems in a single generation pass and only engages more expensive strategies when verification fails. Second, skill-conditioned generation supplies

accumulated solution patterns as structured context for the current problem. The accuracy margin over Reflexion is small (+0.3 points), but the two systems arrive at comparable accuracy through different mechanisms: Reflexion through per-task retry, FlowEvo through structured reuse of prior successes.

Performance across benchmark families. We emphasize that each benchmark is run as an independent experiment from an empty skill bank, and no skills are transferred across domains; the comparison therefore measures coverage across three independently run benchmark families rather than cross-domain transfer by a single persistent bank. Within this setting, no baseline matches FlowEvo within five points on all three benchmarks, and each baseline succeeds within its original design regime and degrades outside it. Reflexion and ExpeL, whose accumulation mechanism is per-task retry or text-level insight extraction, reach 92.7% and 89.0% on HumanEval but only 52.2% and 46.3% on ALFWorld; ALFWorld’s 134 tasks fall into six task types with substantial parametric recurrence across episodes, and neither per-task retry nor text-level insights can directly replay the executable structure that solved earlier variants. AFLOW and ADAS, which search over workflow topology, reach 94.7% and 82.4% on HumanEval but only 59.2% and 53.0% on ALFWorld; a single optimized topology cannot cover all six ALFWorld task types, and workflow graphs tuned on benchmark aggregate performance cannot adapt to step-by-step environmental feedback. FlowEvo avoids both failure modes because its two reuse modes address two regimes: direct skill reuse captures recurring task structure (the dominant regime on ALFWorld), and skill-conditioned generation provides structured context when each task is unique (the relevant regime on HumanEval and GSM8K). The consistently low token cost across benchmarks is a consequence of the same design: recurring tasks bypass exploration, and unique tasks are attempted in a single pass with more expensive strategies engaged only when verification fails.

4.3 Ablation study: three mechanisms

We use ALFWorld to isolate the contribution of each FlowEvo mechanism through controlled ablations. The ablation baseline is ReAct (Yao et al., 2022), which uses interleaved reasoning and acting without any skill accumulation. Starting from ReAct (33.6%), enabling workflow-to-skill compilation alone raises pass rate to 38.8%; further enabling skill-to-workflow feedback brings it to 80.6%; and enabling skill curation on top of these two mechanisms reaches 82.8%. The three contributions are therefore +5.2, +41.8, and +2.2 percentage points respectively. The remainder of this section explains each.

Workflow-to-skill compilation (W2S). Enabling skill compilation without reuse yields a 5.2-point improvement over ReAct (38.8% vs. 33.6%). This gain arises because the compilation process regularizes the agent’s behavior: extracting reusable structure from successful trajectories provides an implicit learning signal that can benefit subsequent episodes through improved auxiliary memories and more stable intermediate representations.

Skill-to-workflow feedback (S2W). Enabling skill reuse on top of compilation produces the dominant improvement, raising pass rate from 38.8% to 80.6% (+41.8 points), and the subsequent addition of skill curation further lifts it to 82.8%. This confirms that the gain is predominantly driven by skill feedback rather than curation. S2W is the core mechanism of FlowEvo: compiled skills are retrieved and either executed directly or injected as structured context for subsequent workflow generation. On ALFWorld, where task types recur with substantial parametric variation across episodes, direct skill reuse is particularly effective. In the full system (reported in Section 4.4), FlowEvo selects the direct-reuse route on 101 of 134 evaluation tasks, achieving a 98% success rate on those cases.

Skill curation. Skill curation contributes a small aggregate gain (+2.2 points) but provides a qualitatively important safeguard against negative transfer. During evaluation, the contrastive evaluation mechanism automatically identified the `pick_two` task type as a case of negative transfer and suppressed the corresponding skill. In 10 skill-guided episodes of this task type the success rate was 1/10, compared with 1/3 in unguided episodes, yielding a

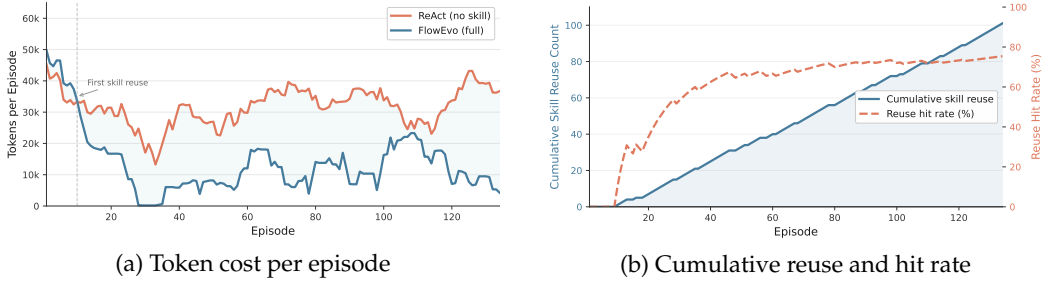


Figure 2: Skill accumulation dynamics on ALFWorld. (a) FlowEvo’s token cost drops sharply once the first compiled skill becomes available around episode 10 (dashed line) and remains substantially below ReAct for the rest of the run. (b) Cumulative direct-reuse count (green) grows nearly linearly to 101/134, and the reuse hit rate (orange) stabilizes around 75%.

contrastive delta of -0.23 that fell below the suppression threshold of -0.1 and triggered automatic suppression. Without curation, this skill would continue to be applied on subsequent `pick_two` episodes and actively harm performance; with curation, it is suppressed automatically and routing defaults to dynamic generation. This episode shows that skill accumulation is not uniformly beneficial and that curation is what keeps the skill bank from amplifying its own worst entries.

4.4 Analysis of skill accumulation dynamics

We further analyze the dynamics of skill accumulation on ALFWorld to understand how FlowEvo’s behavior evolves over the course of evaluation.

Token efficiency. Figure 2(a) shows the per-episode token cost for FlowEvo and ReAct over the 134 evaluation tasks. After the first compiled skill becomes available (around episode 10), FlowEvo’s token consumption drops sharply and remains substantially below ReAct throughout the remaining episodes. On average, FlowEvo uses 12,267 tokens per episode. The four baselines use 29,671 (ADAS), 30,137 (AFLOW), 31,900 (Reflexion), and 32,958 (ExpeL) respectively. The gain comes directly from the skill-to-workflow feedback loop: when a compiled skill is reused directly, the agent bypasses exploratory interaction and proceeds to verification.

Library growth and reuse rate. Figure 2(b) tracks the growth of the skill library over episodes. The cumulative direct-reuse count increases steadily to 101 of 134 evaluation episodes. The reuse hit rate rises quickly during the first 20 episodes as the library accumulates templates for the most common task types, then stabilizes around 75%. Of the 101 direct-reuse episodes, 99 succeed (98% success rate), confirming that compiled skills generalize reliably across parametric variations of the same task type.

Per-task-type improvement. Figure 3 decomposes the ALFWorld pass rate by task type. The magnitude of the improvement on each type tracks how often the compiled skill completed the task through direct execution alone, without fallback to skill-conditioned or dynamic generation (we refer to this as the direct-hit rate). For the two task types with the largest gains, `pick_heat` (+78 points) and `pick_clean` (+64 points), the compiled skill reached a direct-hit rate of 83% and 71% respectively, capturing the full action sequence and generalizing across object variations. `pick_cool` (+57 points) and `pick_place_simple` (+54 points) show intermediate gains and direct-hit rates of 57%–58%. The smallest gain, `look_light` (+28 points), corresponds to a direct-hit rate of 0%: the retrieved skill matched on 15 of 18 episodes, but the router never selected pure direct execution for this task type (Table 5, Direct column = 0/18), and every successful episode was resolved through skill-conditioned generation or, in 3 retrieval-downgraded cases, through pure dynamic generation. This happens because `look_light` tasks exhibit larger configuration variance than `pick` tasks, so the compiled action sequence is judged insufficient for direct replay at routing time yet remains useful as structured context for planning. The +28-point gain is

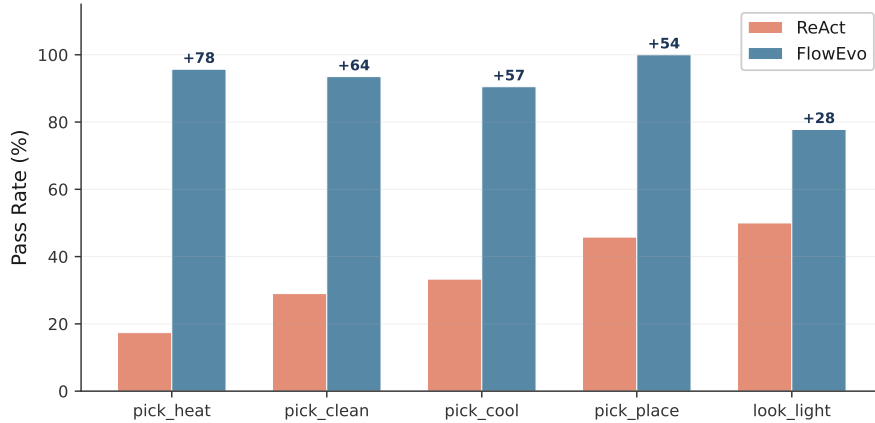


Figure 3: Per-task-type success rate on ALFWorld (GPT-4o-mini; n per type varies). FlowEvo improves over ReAct on all five displayed task types. The magnitude of improvement tracks how often the compiled skill completed the task through direct execution alone without fallback to skill-conditioned or dynamic generation. The pick_two task type is omitted because its compiled skill was suppressed by skill curation as a case of negative transfer (see Section 4.3).

therefore attributable entirely to the second reuse mode, and the five task types together trace an approximately monotone relation between direct-hit rate and gain magnitude (pick_place_simple and pick_cool sit nearly on top of each other, but the overall ordering is preserved).

The sixth task type, pick_two ($n = 17$), is omitted from Figure 3 and analyzed in Section 4.3 instead: its compiled skill was identified by skill curation as harmful and was suppressed during evaluation. It is not a representative “gain” case.

5 Discussion

The results in Table 2 and the ALFWorld ablations (Section 4.3) suggest that accumulated skills help through two complementary mechanisms. The dominant effect is direct skill reuse: on ALFWorld, compiled skills are replayed on 101 of 134 episodes with a 98% success rate, accounting for the bulk of the +41.8-point improvement attributable to skill-to-workflow feedback. The second effect is skill-conditioned workflow generation, where retrieved skills serve as structured context even when direct replay is unavailable; this mode is particularly relevant on code and math benchmarks, where each task is unique and exact reuse is rare. Together, these two modes allow a single framework to achieve strong results across both interactive environments and code/math generation—task families that no single baseline covers.

These results also help clarify the relation between workflow optimization and skill learning. A successful workflow is usually richer than a short text memory, but it is also more reusable than a one-off trace. FlowEvo treats part of that workflow as a persistent executable object with an interface, validation tests, and usage history. This gives the agent a way to keep useful execution structure after the original task has ended. The point is not to turn every successful run into a new skill, but to preserve the stable and reusable parts of successful workflows in a form that later episodes can call, inspect, and manage.

The current evidence should guide how the results are interpreted. Our experiments rely on domains where correctness can be checked with environment feedback (ALFWorld) or hidden tests (code and math benchmarks). This makes skill admission and curation easier to study, but broader environments may introduce different challenges, especially when feedback is delayed, partial, or noisy, as observed in AI-assisted scientific evaluation settings (Wang et al., 2025a). Additionally, the framework requires the base model to have sufficient task-solving capability to bootstrap the compile-reuse loop; on environments

where the base model rarely succeeds, the skill bank may remain empty. A natural next step is to test whether the same workflow-to-skill loop remains effective when verification is weaker and when task distributions shift over longer horizons.

Two future directions appear particularly important. First, the current system mainly evolves task-solving skills themselves. It is worth asking whether self-evolution can extend to other parts of the agent harness, such as verification helpers, retry policies, retrieval heuristics, or lightweight control components around the main solver. Connecting the skill bank to broader tool ecosystems through standardized protocols (Ouyang et al., 2025; Di et al., 2026) is another promising avenue, allowing accumulated skills to interoperate with externally curated tools and domain-specific scientific resources. Second, long-running use turns skill-bank maintenance into a central problem. As the bank grows, the system must decide how to revise, merge, retire, and organize skills while limiting retrieval interference and stale behavior. In that sense, continual use may depend not only on how new skills are acquired, but also on how the skill bank is maintained over time (Yue et al., 2026b).

6 Conclusion

FlowEvo is a training-free framework that compiles successful workflows into reusable skill records. Each record pairs a callable artifact with auxiliary structured guidance, and admission applies interface, replay, and safety checks where feasible. These skill records are then used to improve later problem solving. Across benchmarks spanning interactive environments and code/math generation, FlowEvo achieves the best accuracy-cost tradeoff among the evaluated baselines under our implementation settings. Controlled ablations confirm that each of the three mechanisms, workflow-to-skill compilation, skill-to-workflow feedback, and skill curation, contributes to the overall result on ALFWorld. These findings support a simple view: agents can improve over time by retaining and managing reusable execution structure at inference time.

References

- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, et al. Cua-skill: Develop skills for computer using agent. *arXiv preprint arXiv:2601.21123*, 2026.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Shimin Di, Xujie Yuan, Hanghui Guo, Chaoqian Ouyang, Zhangze Chen, Ling Yue, Libin Zheng, Jia Zhu, Shaowu Pan, Jian Yin, et al. Toolrosetta: Bridging open-source repositories and large language model agents through automated tool standardization. *arXiv preprint arXiv:2603.09290*, 2026.
- Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, Bryan Hooi, Min Lin, and Tianyu Pang. Flowreasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2024. URL <https://arxiv.org/abs/2408.08435>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al.

- Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Chaoqian Ouyang, Ling Yue, Shimin Di, Libin Zheng, Linan Yue, Shaowu Pan, Jian Yin, and Min-Ling Zhang. Code2mcp: Transforming code repositories into mcp services. *arXiv preprint arXiv:2509.05941*, 2025.
- Shaowu Pan, Ling Yue, Nithin Somasekharan, Tingwen Zhang, Yadi Cao, Zhangze Chen, and Shimin Di. Automating computational fluid dynamics with llm-based multi-agent systems. 2026.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. Clin: A continually learning language agent for rapid task adaptation and generalization. *arXiv e-prints*, pp. arXiv–2310, 2023.
- Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent Q: Advanced reasoning and learning for autonomous ai agents, 2024. URL <https://arxiv.org/abs/2408.07199>.
- Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 6922–6939, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.
- Theodore Sumers, Shunyu Yao, Karthik R Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.
- Ye Tian, Baolin Peng, Linfeng Song, Lifeng Jin, Dian Yu, Lei Han, Haitao Mi, and Dong Yu. Toward self-improvement of llms via imagination, searching, and criticizing. *Advances in Neural Information Processing Systems*, 37:52723–52748, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Jialiang Wang, Yuchen Liu, Hang Xu, Kaichun Hu, Shimin Di, Wangze Ni, Linan Yue, Min-Ling Zhang, Kui Ren, and Lei Chen. When ai reviews science: Can we trust the referee? 2025a.
- Jialiang Wang, Shengxiang Xu, Hanmo Liu, Jiachuan Wang, Yuyu Luo, Shimin Di, Min-Ling Zhang, and Lei Chen. Learning to compose for cross-domain agentic workflow generation. *arXiv preprint arXiv:2602.11114*, 2026a.
- Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025b.

- Yinjie Wang, Ling Yang, Guohao Li, Mengdi Wang, and Bryon Aragam. Scoreflow: Mastering llm agent workflows via score-based preference optimization. *arXiv preprint arXiv:2502.04306*, 2025c.
- Yishu Wang, Wei Liu, Yifan Li, Shengxiang Xu, Xujie Yuan, Ran Li, Yuyu Luo, Jia Zhu, Shimin Di, Min-Ling Zhang, et al. Peromas: A multi-agent system of perovskite material discovery. *arXiv preprint arXiv:2602.13312*, 2026b.
- Zhiruo Wang, Daniel Fried, and Graham Neubig. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks. *arXiv preprint arXiv:2401.12869*, 2024a.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024b. URL <https://arxiv.org/abs/2409.07429>.
- Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. Tool-Genesis: A task-driven tool creation benchmark for self-evolving language agent, 2026a. URL <https://arxiv.org/abs/2603.05578>.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026b.
- Shengxiang Xu, Jiayi Zhang, Shimin Di, Yuyu Luo, Liang Yao, Hanmo Liu, Jia Zhu, Fan Liu, and Min-Ling Zhang. Robustflow: Towards robust agentic workflow generation. *arXiv preprint arXiv:2509.21834*, 2025a.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025b.
- Liang Yao, Shengxiang Xu, Fan Liu, Chuanyi Zhang, Bishun Yao, Rui Min, Yongjun Li, Chaoqian Ouyang, Shimin Di, and Min-Ling Zhang. Remoteagent: Bridging vague human intents and earth observation with rl-based agentic mllms. *arXiv preprint arXiv:2604.07765*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R Fung, Hao Peng, and Heng Ji. Craft: Customizing llms by creating and retrieving from specialized toolsets. *arXiv preprint arXiv:2309.17428*, 2023.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2024. URL <https://arxiv.org/abs/2401.10020>.
- Linan Yue, Yichao Du, Yizhi Wang, Weibo Gao, Fangzhou Yao, Li Wang, Ye Liu, Ziyu Xu, Qi Liu, Shimin Di, et al. Don't overthink it: A survey of efficient rl-style large reasoning models. *arXiv preprint arXiv:2508.02120*, 2025.
- Ling Yue, Kushal Raj Bhandari, Ching-Yun Ko, Dhaval Patel, Shuxin Lin, Nianjun Zhou, Jianxi Gao, Pin-Yu Chen, and Shaowu Pan. From static templates to dynamic runtime graphs: A survey of workflow optimization for llm agents. *arXiv preprint arXiv:2603.22386*, 2026a.
- Ling Yue, Ching-Yun Ko, Pin-Yu Chen, Shimin Di, and Shaowu Pan. Building mcp-native hierarchical ai scientist ecosystems: A perspective on scaling multi-agent scientific discovery. 2026b.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation, 2024. URL <https://arxiv.org/abs/2410.10762>.
- Qizheng Zhang, Michael Wornow, Gerry Wan, and Kunle Olukotun. Agentic plan caching: Test-time memory for fast and cost-efficient llm agents. *arXiv preprint arXiv:2506.14852*, 2025b.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory, 2026a. URL <https://arxiv.org/abs/2601.03192>.
- Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. Live-Evo: Online evolution of agentic memory from continuous feedback, 2026b. URL <https://arxiv.org/abs/2602.02369>.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025c.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. SkillWeaver: Web agents can self-improve by discovering and honing skills, 2025a. URL <https://arxiv.org/abs/2504.07079>.
- Chengqi Zheng, Jianda Chen, Yueming Lyu, Wen Zheng Terence Ng, Haopeng Zhang, Yew-Soon Ong, Ivor Tsang, and Haiyan Yin. Mermaidflow: Redefining agentic workflow generation via safety-constrained evolutionary programming. *arXiv preprint arXiv:2505.22967*, 2025b.
- Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, et al. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*, 2023.
- Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, et al. A survey of data agents: Emerging paradigm or overstated hype? *arXiv preprint arXiv:2510.23587*, 2025.

A LLM Prompts

This appendix lists the prompts used by the core FlowEvo pipeline on ALFWorld (action generation and trace-to-skill compilation) and on the code/math benchmarks (generation and repair). Prompts for external baselines that we reimplemented (Reflexion, Expel, AFLOW, ADAS) are omitted here; they are provided in the released code. All prompts are shown with placeholders in `{curly_braces}` denoting runtime substitutions.

A.1 ALFWorld action generation

Per-step action generation in `AlfWorldGenerator.step` (`src/alfworld_/generator.py:93`), called with `temperature=0.0`, `max_output_tokens=256`. The system prompt is a fixed string; the user prompt is assembled per step with the current observation, action history, and retrieved skill context.

Listing 1: ALFWorld step system prompt.

```
You are an expert household robot completing tasks in a virtual home. You will be
  ↪ given a task goal, the current observation, and a list of valid actions.

At each step:
1. Think about what you need to do next and why.
2. Choose exactly ONE action from the valid actions list.

Common task patterns:
- pick_and_place: go to object location -> take it -> go to destination -> put it
- pick_clean_then_place: go to object -> take -> go to sinkbasin -> clean -> go to
  ↪ dest -> put
- pick_heat_then_place: go to object -> take -> go to microwave -> heat -> go to dest
  ↪ -> put
- pick_cool_then_place: go to object -> take -> go to fridge -> cool -> go to dest ->
  ↪ put
- examine_in_light: go to object -> take -> go to lamp -> use lamp
- pick_two: find first object -> take -> go to dest -> put -> find second -> take ->
  ↪ go to dest -> put

Format your response as:
Think: <your step-by-step reasoning>
Act: <the exact action from the valid actions list>
```

Listing 2: ALFWorld step user prompt template. The `{skill_context}` block is empty in dynamic mode and carries retrieved skill content (exemplars, insights, or full template) in skill-conditioned mode.

```
Task: {task_goal}

Relevant experience:
{skill_context}

Recent actions:
> {action_history[i]}
  {observation_history[i][:120]}
...

Current observation:
{observation}

Valid actions ({len}):
{cmd_1}
{cmd_2}
...

Choose ONE action from the valid actions list above:
```

A.2 ALFWorld trace-to-skill compilation (Layer 2: guideline)

Invoked by `AlfWorldCompiler._compile_guideline` (`src/alfworld/_compiler.py:234`) on successful traces. Settings: `temperature=0.0`, `max_output_tokens=500`.

Listing 3: Guideline compilation system prompt.

You are a task analysis assistant. Extract concise procedural rules.

Listing 4: Guideline compilation user prompt template. Abstractness constraints (no object names, no numbers, generic terms) are enforced in the prompt itself rather than by post-processing.

You are analyzing a successful task execution trace to extract reusable procedural
 $\hookrightarrow$ rules.

Task type: {task_type}
 Task goal: {goal}

Successful action sequence:
 1. {action} $\rightarrow$ {observation[:60]}
 2. {action} $\rightarrow$ {observation[:60]}
 ...

Extract 3-5 reusable procedural rules that would help an agent solve similar tasks in
 $\hookrightarrow$ the future.

Requirements:

- Each rule must be ABSTRACT: do NOT include specific object names, location names,
 $\hookrightarrow$ or numbers
- Use generic terms like "target object", "cleaning station", "destination receptacle
 $\hookrightarrow$ "
- Each rule should be one sentence in the format: "When [situation], [do what],
 $\hookrightarrow$ because [why]"
- Focus on STRATEGY (search approach, action ordering) not exact steps
- Include at least one rule about what to AVOID (common failure patterns)
- 3-5 rules only, keep each concise

Rules:

A.3 Code and math generation with skill context

Primary generation in the code/math runner (`src/code_math/runner.py:506`) is a single call parameterized by `_GEN_SETTINGS = (temperature=0.0, max_output_tokens=2048)` at Level 1 and `_GEN_L2 = (0.5, 2048)` for the 2-candidate mini-ensemble at Level 2. Skill-conditioned generation uses the same call with a retrieved-skill block prepended to the user prompt:

Listing 5: Code task skill-context prefix (prepended to the standard HumanEval problem statement). For math tasks the prefix reads "Here is a similar solved problem for reference:".

[Reference from similar solved problem:
 {skill_context}
]

A.4 Reflexion retry

ALFWorld reflexion is triggered on episode failure (`src/alfworld/_runner.py`, `temperature=0.0`, `max_output_tokens=200`). The reasoning output is appended to subsequent retries as an additional system directive.

Listing 6: ALFWorld reflexion prompts.

System: You are a task reflection assistant.

User:

You attempted a household task and failed.

Task: {task.goal}

Actions taken (last 10): {ref_actions}

Reflect on what went wrong and what to do differently. Be specific and concise (1-2
 $\hookrightarrow$ sentences).

The code/math reflexion prompt (src/code_math/runner.py:586) conditions additionally on retrieved skill context and the failed test output; for brevity we omit the full template here and refer to the released code.

B Framework Thresholds and Configuration Constants

FlowEvo’s routing, admission, and curation decisions are governed by a set of numeric thresholds. Table 3 lists the values that affect the results reported in Section 4. The full set of 177 constants appears in the released code.

Table 3: Core decision thresholds used in FlowEvo. All values are defaults; no per-benchmark tuning was performed for the reported results.

Constant	Value
<i>Retrieval</i>	
top_k	3
retrieval_threshold	5.0
Historical positive-transfer cap	2.5
Negative-transfer risk cap	3.0
<i>Routing</i>	
_COMPATIBILITY_DIRECT_THRESHOLD	0.55
Direct-execute compat gate	≥ 1.5
_CODE_EXCERPT_THRESHOLD	4.6
Max seed context skills	2
<i>Admission</i>	
banned_imports	{os, subprocess, socket, requests, pathlib}
banned_calls	{eval, exec, compile, __import__, open}
prune_audit_fail_threshold	2
<i>Curation (ALFWorld)</i>	
_CONTRASTIVE_MIN_GUIDED	5
_CONTRASTIVE_MIN_UNGUIDED	3
_CONTRASTIVE_HARM_THRESHOLD	-0.1
_AUDIT_MIN_USES	3
_AUDIT_UTILITY_THRESHOLD	0.5
_AUDIT_INACTIVITY_LIMIT	50 episodes
<i>Adaptive escalation</i>	
Level 1 (greedy)	temp=0.0, max=2048
Level 2 (mini-ensemble, 2 candidates)	temp=0.5, max=2048
Level 3 (retry, hot)	temp=0.7, max=2048
Level 4 (retry, cool)	temp=0.2, max=2048
Reflexion reasoning	temp=0.0, max=300
<i>Execution</i>	
ALFWorld max_steps	50
Sandbox timeout (code/math)	10s
Draft temperature / max tokens	0.2 / 900
Repair temperature / max tokens	0.15 / 900

The contrastive harm threshold -0.1 combined with the minimum-samples gates ($_CONTRASTIVE_MIN_GUIDED = 5$, $_CONTRASTIVE_MIN_UNGUIDED = 3$) determines when skill curation suppresses a skill: a candidate is suppressed only after at least 5 guided and 3 unguided samples, and only if the delta falls below -0.1 . The suppression event reported in Section 4.3 (delta -0.23 , guided $1/10$ vs. unguided $1/3$) satisfies these gates: $10 \geq 5$, $3 \geq 3$, and $0.1 - 0.333 = -0.233 < -0.1$.

C Skill Record Example and Bank Composition

This appendix shows a complete active skill record from the ALFWorld run, followed by the final composition of the skill bank. The implementation organizes each skill record into three layers: a *Layer-1 template* that captures the parameterized action sequence (Listing 7), a *Layer-2 guideline* produced by the compilation prompt of Appendix A.2, and a *Layer-3 insight* that aggregates environment priors across traces of the same task type (Listing 8). We show Layer-1 and Layer-3 here; Layer-2 guidelines are generated directly from the prompt in Appendix A.2 and are not reproduced.

C.1 A compiled skill: pick_heat_then_place_in_recep

Listing 7: Layer-1 template record for pick_heat_then_place_in_recep. This is the on-disk representation used at retrieval time.

```
{
  "template": {
    "template_id": "alf_skill_pick_heat_then_place_in_recep_112be592",
    "task_type": "pick_heat_then_place_in_recep",
    "action_template": [
      "go to {object_location}",
      "take {object} from {object_location}",
      "go to {heating_station}",
      "heat {object} with {heating_station}",
      "go to {target_location}",
      "move {object} to {target_location}"
    ],
    "parameters": ["heating_station", "object",
                  "object_location", "target_location"],
    "example_bindings": {
      "heating_station": "microwave 1",
      "object": "cup 1",
      "object_location": "cabinet 3",
      "target_location": "coffeemachine 1"
    },
    "source_task_id": "eval_out_of_distribution_16",
    "source_trace_id": "alf_ff42ccc3",
    "success_count": 1,
    "failure_count": 0
  },
  "utility": 1.0,
  "status": "active",
  "use_count": 21,
  "success_count": 21,
  "failure_count": 0,
  "last_used_episode": 134,
  "contrastive_stats": {
    "guided_success": 2, "guided_total": 2,
    "unguided_success": 3, "unguided_total": 3
  },
  "injection_suppressed": false
}
```


Key FlowEvo-specific fields: `action_template` is the parameterized action sequence compiled from the source trace, with slots bound at runtime from goal parsing; `utility` is `success_count / use_count` (no smoothing); `status` $\in \{\text{active}, \text{suppressed}\}$ is set by curation; `contrastive_stats` track a balanced matched subset of guided vs. unguided episodes used for harm-delta estimation, and are therefore typically smaller than `use_count`. The insight record below is retrieved jointly with the template and injected as structured context when direct execution is not selected.

Listing 8: Layer-3 insight record accompanying the template above. Insights are generated after `_INSIGHT_MIN_SAMPLES = 3` traces of the same task type and provide environment priors that generalize across parametric variation.

```
{
  "task_type": "pick_heat_then_place_in_recep",
  "search_priority": "countertop > cabinet > coffeemachine > garbagecan > microwave",
  "common_objects": [],
  "common_locations": ["countertop", "cabinet",
                      "coffeemachine", "garbagecan", "microwave"],
  "common_pitfalls": ["100% of failures are timeouts -- explore more efficiently"],
  "environment_facts": ["microwave is usually numbered 1"],
  "sample_count": 23
}
```

Note that the insight is aggregated over *all* `sample_count = 23` traces of this task type, including the one episode that was routed through `pure_dynamic` and failed (see Table 5). The `common_pitfalls` field therefore describes a failure observed in the non-skill path; it is consistent with the Layer-1 template record reporting zero failures over its own 21 uses.

C.2 Final skill-bank composition

Table 4 summarizes the bank at the end of the 134-episode ALFWorld run. Six templates are compiled, one per task type; the `pick_two_obj_and_place` template is suppressed by curation as discussed in Section 4.3.

Table 4: Active skills after 134 ALFWorld episodes. **Uses** counts episodes in which the skill was retrieved above `retrieval_threshold` and thus participated in routing (whether the router eventually selected direct execution or skill-conditioned generation, or downgraded to dynamic). **Success / Fail** are the outcomes of those retrieval-matched episodes; **Utility** = `Success / Uses`. Entries are cumulative over the run. For five of the six task types, **Uses** equals `Direct + Cond` from Table 5; `look_at_obj_in_light` has 3 additional retrieval matches that were routing-downgraded to `pure_dynamic` (see Section 4.4: "the retrieved skill matched on 15 of 18 episodes").

Task type	Status	Uses	Success	Fail	Utility
<code>pick_and_place_simple</code>	active	23	23	0	1.00
<code>pick_clean_then_place_in_recep</code>	active	28	28	0	1.00
<code>pick_heat_then_place_in_recep</code>	active	21	21	0	1.00
<code>pick_cool_then_place_in_recep</code>	active	20	18	2	0.90
<code>look_at_obj_in_light</code>	active	15	13	2	0.87
<code>pick_two_obj_and_place</code>	suppressed	10	1	9	0.10

The `pick_two_obj_and_place` row reflects the state at the end of the run: ten guided episodes accumulated before suppression (matching the "10 skill-guided episodes" in Section 4.3 and `Cond = 10` in Table 5), with one success and nine failures, yielding a final utility of $1/10 = 0.10$. After suppression, subsequent `pick_two_obj_and_place` episodes bypassed the bank entirely and are *not* counted in this row (they contribute to the `Dyn` column of Table 5).

D Per-Task-Type Routing Distribution

Table 5 breaks down the 134 ALFWorld episodes by the router decision actually taken. Three internal modes are distinguished: `direct_skill` (template called as an executable subroutine, no fallback), `direct_then_*` (template invoked but execution required fallback to skill-conditioned or dynamic generation), and `exemplar_guided` / `pure_dynamic` (no template invocation).

Terminology mapping to the main text. The main text uses a coarser classification:

- “Direct-reuse route” = `direct_skill` $\cup$ `direct_then_*`: 101/134 episodes with 99 successes (98%), as reported in Section 4.4 and plotted as the reuse curve in Figure 2(b).
- “Skill-conditioned generation” = `exemplar_guided` and related skill-context-only variants: retrieved skills supplied as structured context without template invocation.
- “Dynamic generation” = `pure_dynamic`: no retrieval is used.

In Table 5, the **Direct** column reports `direct_skill` only; the **Cond.** column collapses `direct_then_*` and `exemplar_guided` together. The paper’s “direct-reuse” count of 101 therefore equals **Direct** (67) + the `direct_then_*` subset of **Cond.** (34), rather than **Direct** alone.

Table 5: Internal routing modes by task type on ALFWorld. “Direct” = pure `direct_skill`; “Cond.” = `direct_then_*` plus `exemplar_guided`; “Dyn.” = `pure_dynamic`. “Direct SR” is the success rate among pure direct-execute episodes. “Overall SR” is the pass rate on all episodes of the task type.

Task type	<i>n</i>	Direct	Cond.	Dyn.	Direct SR	Overall SR
look_at_obj_in_light	18	0	12	6	—	14/18 (78%)
pick_and_place_simple	24	14	9	1	14/14	24/24 (100%)
pick_clean_then_place_in_recep	31	22	6	3	22/22	29/31 (94%)
pick_cool_then_place_in_recep	21	12	8	1	12/12	19/21 (90%)
pick_heat_then_place_in_recep	23	19	2	2	19/19	22/23 (96%)
pick_two_obj_and_place	17	0	10	7	—	3/17 (18%)
Total	134	67	47	20	67/67	111/134 (83%)

Curation suppression event. One suppression event was triggered during the run, satisfying the curation gates of Section B:

- `pick_two_obj_and_place`: guided 1/10 (10%) vs. unguided 1/3 (33%), $\Delta = -0.23$; template suppressed (the -0.23 case analyzed in detail in Section 4.3).

After suppression, the remaining four `pick_two_obj_and_place` episodes routed through `pure_dynamic` (contributing to the Dyn count of $7 = 3$ pre-suppression + 4 post-suppression in Table 5); they did not use the skill bank and are therefore excluded from the Uses count of Table 4.